

Proving the Capla version of `htr3.c`

The plan, cut into tasks for separate agents

`code/APaul/capla/htr3/htr3.b` is the search of `code/APaul/htr3_new.c` in Capla, for one line of a table. The goal is a Rocq theorem saying that it returns the hard-to-round cases: **on a line that satisfies the six conditions (`line_ok` of `code/exptable9`), every input $x_0 + ju$, $0 \leq j < n$, with $\exp(x_0 + ju)/v$ within 2^{-m} of an integer is in the output of `search`**. The search walks $j = 0, \dots, n - 1$, the half-open $[x_0, x_1)$ of `htr3_new.c`. The output may hold more candidates; the theorem is that none is missed.

1 What is assumed

- **WP_sound**, admitted in Capla itself (`bfrontend/WP.v`): every result on a Capla program goes through Capla's weakest-precondition calculus, whose soundness is not proved yet.
- **The C driver `main_htr3.c`**: reading the file, $u = \text{ulp}(x_0)$ by `frexpr`, printing $x_0 + ju$. The theorem is stated on the numbers of a line; that the text of a line means these numbers is `parse_sound` of `code/exptable9`, for the Rocq parser, not for the C one.
- **The capacity of the output**: `search` writes the first `cap` candidates and returns their number; the theorem assumes the number is at most `cap`.

2 The program, cut into functions

`htr3.b` is rewritten as five functions, each proved on its own; the rewritten program must give the same output as `htr3.c` on the tests of the README.

function	what it does, on the flat array B of $k\ell$ words
<code>addw B m ia ib l</code>	$B_{ia} \leftarrow B_{ia} + B_{ib} \bmod \beta^\ell$ (words $[ia, ia + \ell)$)
<code>subw B m ia ib l</code>	$B_{ia} \leftarrow B_{ia} - B_{ib} \bmod \beta^\ell$
<code>diffstab B m k l</code>	for $i = 1..k - 1$, for $j = k - 1$ down to i : <code>subw j, j - 1</code>
<code>tstep B m k l</code>	for $t = 0..k - 2$: <code>addw t, t + 1</code> (<code>step</code> is a Capla keyword)
<code>search B m k l n err out cap</code>	<code>diffstab</code> ; top word of $B_0 += \text{err}$; for $j = 0..n - 1$: test, record, <code>tstep</code>

3 The tasks

Each task is one file with fixed statements (written before the task starts, with **Admitted**); the agent replaces the **Admitted** by proofs and may use the statements of the other tasks, not their proofs. All Capla files build in the opam switch `capla` (Rocq 9.0); with `coq-lsp` installed there, `rocq-mcp` can drive them after `rocq_switch`.

task	file	proves	needs
T0	<code>HtrBase.v</code> , <code>htr3.v</code>	setup (done before the others): the program as a Rocq term, the common definitions (<code>val</code> , the coefficient B_i of a flat array, $M = \beta^\ell$), the list and carry lemmas of <code>add_proof.v</code>	–
T1	<code>AddwProof.v</code>	<code>addw_spec</code> : the words of B_{ia} become $(B_{ia} + B_{ib}) \bmod M$, the others are unchanged	T0
T2	<code>SubwProof.v</code>	<code>subw_spec</code> : the same with –	T0
T3	<code>DiffstabProof.v</code>	<code>diffstab_spec</code> : the coefficients become the differences of the input ones, modulo M (<code>diffstabZ</code> , a plain function on <code>list Z</code> defined in T0)	T0, T2

T4	<code>TstepProof.v</code>	<code>tstep_spec</code> : the coefficients become <code>tstepZ</code> of them modulo M ($B_t + B_{t+1}$, the last one unchanged)	T0, T1
T5	<code>SearchProof.v</code>	<code>search_spec</code> : the output is the list of the $j < n$ whose top word of B_0 , after <code>diffstabZ</code> , the window and j <code>tstepZ</code> , is at most 2 err	T0, T3, T4
T6	<code>HtrMath.v</code>	no Capla: (a) <code>diffstabZ</code> of $P(0), \dots, P(k-1)$ is the table of differences of P modulo M , and j <code>tstepZ</code> give $P(j)$ in B_0 (from <code>Shift.v</code>); (b) the test of <code>hscan</code> implies the top-word test (<code>hscan_exp</code> scans $j < n$, as the search does); written in <code>code/APaul/rocq</code> next to <code>TaylorLink.v</code> , in the switch <code>native</code> , and carried over by T7	T0
T7	port	<code>code/APaul/rocq/{Shift, TaylorScan, TaylorReal, TaylorLink}.v</code> , <code>HtrMath.v</code> and the proof files of <code>code/exptable9</code> build in the switch <code>capla</code> (Rocq 9.0)	–, then T6
T8	<code>HtrFinal.v</code>	the theorem of the introduction: from <code>line_ok</code> and the hard input, j is in the output of <code>search</code>	T5, T6, T7

T1, T2, T6 and T7 can start at once, after T0; then T3 and T4; then T5; then T8.

4 What each statement says

To be fixed in T0, before the agents start:

- the numbers of a flat array: `coef B i` is the list of the ℓ words of B_i , and `vcoef B i := val (coef B i)` its value, least significant word first;
- `diffstabZ` and `tstepZ` on list Z , and the top word `top b := (b mod M) / 2^(64 (ell - 1))`;
- for the Capla functions, the shape of `add_spec` (`code/APaul/capla/htr3/proof/add_proof.v`): from `eval_funcall ge (Internal f) args e1 (Some result)`, the final array in `e1`, its length unchanged, and the numbers it holds.

5 Order of the work

1. T0, by hand: the cut of `htr3.b`, its tests, the generated `htr3.v`, `HtrBase.v` and the stub files, all compiling in the switch `capla`.
2. Four agents in parallel: T1, T2, T6, T7.
3. Then T3 and T4 in parallel, then T5, then T8.

Estimated, not measured: T1 and T2 are close to the proof of `add` (91 lines and reusable lemmas); T3 and T5 are the larger ones (nested loops, calls to proved functions, as `mul_basecase` in Capla's examples); T6 and T8 are mostly statements.